Lab-06 Kafka Integration with C# Hands-On

Scenario

The objective of this hands-on exercise was to integrate Apache Kafka with C# and demonstrate message streaming through Kafka. Kafka was executed using Docker, and C# applications were created to publish and consume chat messages.

Objective

To run Kafka, create a topic, send messages using a producer, consume messages using a consumer, and integrate Kafka with C# using the Confluent.Kafka package.

Implementation

Docker Desktop was started and verified successfully.

Kafka was started using Docker with the Apache Kafka image.

The Kafka container was created and port **9092** was exposed for communication.

The Kafka container was verified using the docker ps command.

A Kafka topic named **chat-message** was created inside the Kafka container.

A Kafka console consumer was started and subscribed to the **chat-message** topic.

A Kafka console producer was started and connected to the same topic.

Messages were entered in the producer command prompt.

The same messages were received and displayed in the consumer command prompt.

This confirmed that Kafka was running successfully and message streaming was working.

After completing Kafka command-line testing, C# integration was implemented.

A C# Console Application named **KafkaChatProducer** was created.

The **Confluent.Kafka** NuGet package was installed.

The producer application was configured with:

BootstrapServers = localhost:9092
Topic = chat-message

The C# producer accepted chat messages from the console and published them to Kafka.

A second C# Console Application named **KafkaChatConsumer** was created.

The **Confluent.Kafka** NuGet package was installed in the consumer project.

The consumer was configured with:

BootstrapServers = localhost:9092
GroupId = chat-consumer-group
Topic = chat-message

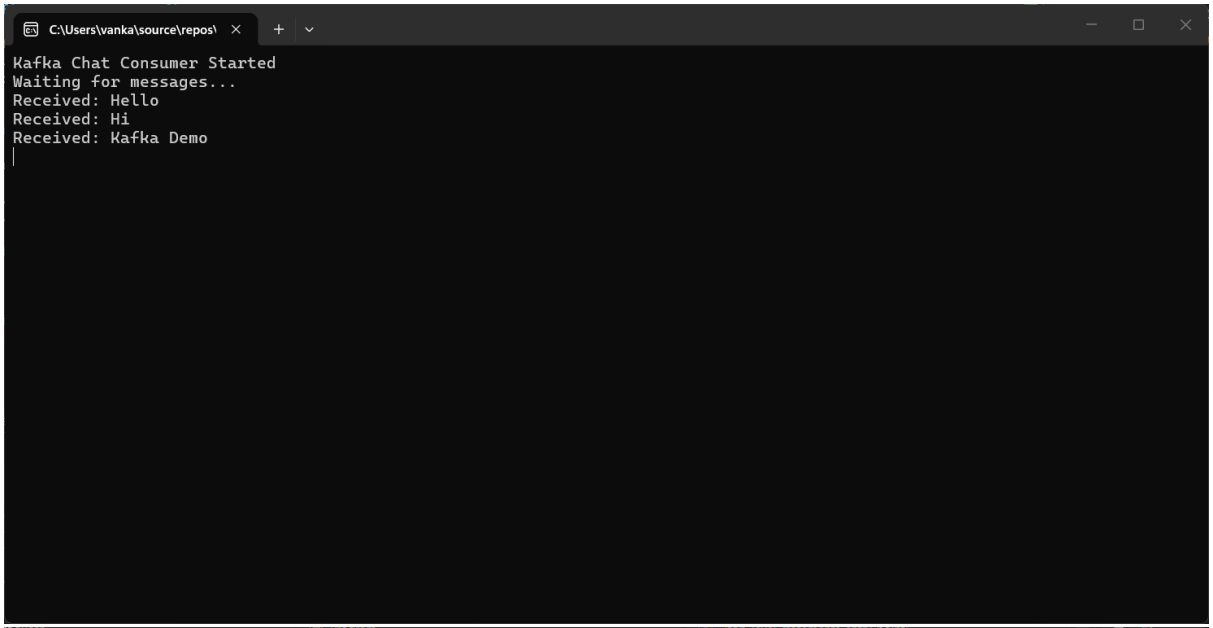
The C# consumer subscribed to the Kafka topic and continuously listened for messages.

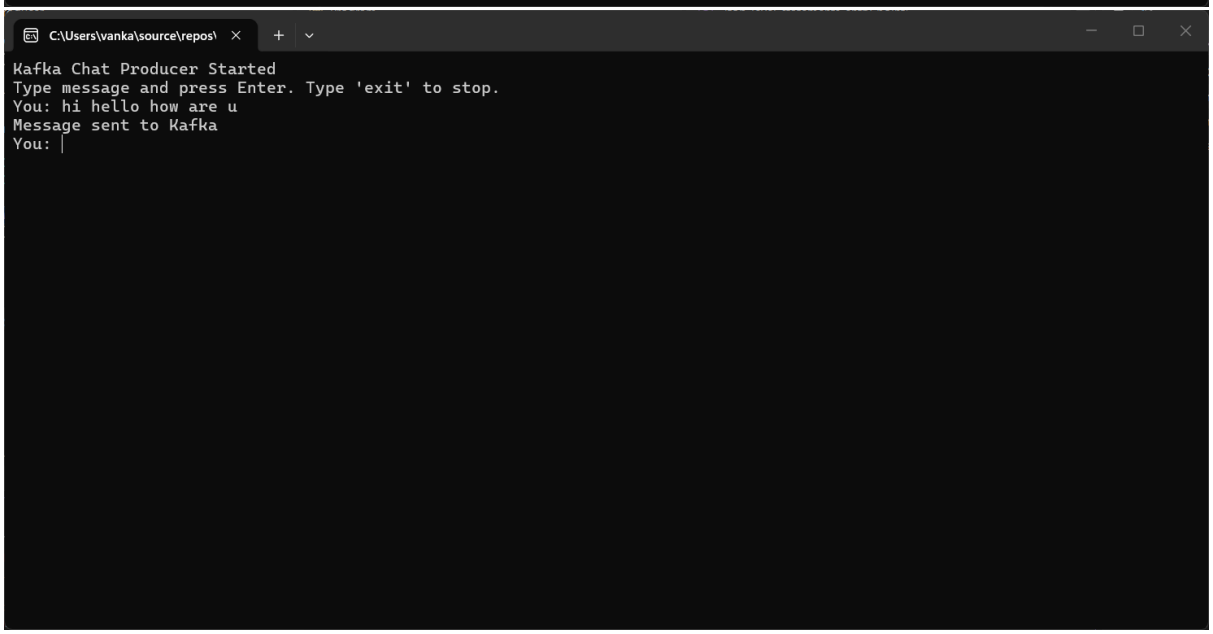
Both C# applications were executed.

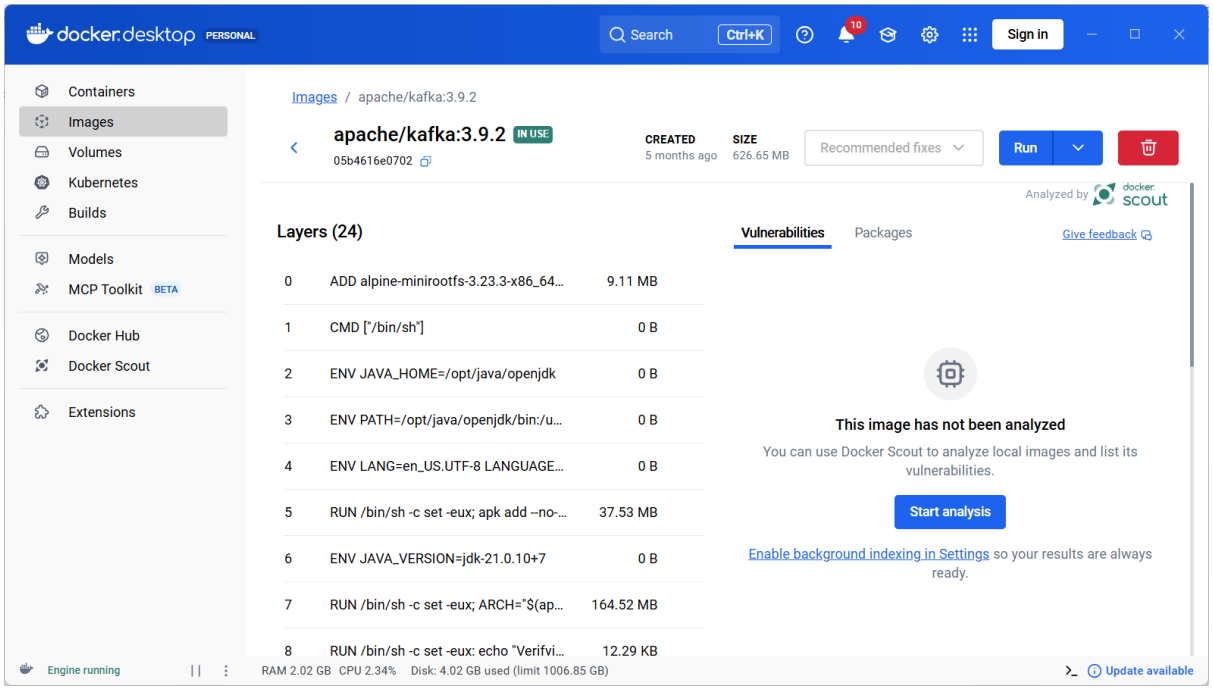
Messages sent from the C# producer were successfully received by the C# consumer.

Result

Kafka was successfully executed using Docker, and the topic **chat-message** was created. Messages were tested using Kafka command-line producer and consumer. After that, Kafka was integrated with C# using the **Confluent.Kafka** package. The C# producer successfully published chat messages, and the C# consumer received them in real time. This completed the Kafka integration hands-on using Docker and C#.






docker desktop PERSONAL
Search Ctrl+K
Sign in
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Images / apache/kafka:3.9.2
apache/kafka:3.9.2 IN USE
05b4616e0702
CREATED
5 months ago
SIZE
626.65 MB
Recommended fixes
Run
Analyzed by docker scout
Layers (24)
Vulnerabilities
Packages
Give feedback
0 ADD alpine-minirootfs-3.23.3-x86_64... 9.11 MB
1 CMD ["/bin/sh"] 0 B
2 ENV JAVA_HOME=/opt/java/openjdk 0 B
3 ENV PATH=/opt/java/openjdk/bin:/u... 0 B
4 ENV LANG=en_US.UTF-8 LANGUAGE... 0 B
5 RUN /bin/sh -c set -eux; apk add --no-... 37.53 MB
6 ENV JAVA_VERSION=jdk-21.0.10+7 0 B
7 RUN /bin/sh -c set -eux; ARCH="$(ap... 164.52 MB
8 RUN /bin/sh -c set -eux: echo "Verifvi... 12.29 KB
This image has not been analyzed
You can use Docker Scout to analyze local images and list its vulnerabilities.
Start analysis
Enable background indexing in Settings so your results are always ready.
Engine running
RAM 2.02 GB CPU 2.34% Disk: 4.02 GB used (limit 1006.85 GB)
Update available